



TECHNISCHE UNIVERSITÄT
BERGAKADEMIE FREIBERG

The University of Resources. Since 1765.

A report on

Image convolution with MPI

Programming Project | Winter Semester 2020-2021

Venkata Mukund Kashyap Yedunuthala

Computational Materials Science

Matriculation register: 65 163

September 15, 2021

Eidesstattliche Erklärung

Ich versichere, dass ich diese Arbeit selbstständig verfasst und keine anderen Hilfsmittel als die angegebenen benutzt habe. Die Stellen der Arbeit, die anderen Werken dem Wortlaut oder dem Sinn nach entnommen sind, habe ich in jedem einzelnen Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht. Diese Versicherung bezieht sich auch auf die bildlichen Darstellungen.

15. September 2021

Venkata Mukund Kashyap Yedunuthala

Declaration

I hereby declare that I completed this work without any improper help from a third party and without using any aids other than those cited. All ideas derived directly or indirectly from other sources are identified as such. This declaration also refers to the representation of figures and visual material.

September 15, 2021

Venkata Mukund Kashyap Yedunuthala

Contents

1	Introduction	5
2	Convolution kernels	5
2.1	Gaussian Blur	5
2.2	Sharpen	5
2.3	Edge Detection	6
3	MPI Parallelization and Communication	6
4	Results	7

List of Figures

1	Distribution of data between processes	6
2	512 x 512 grayscale image	7

1 Introduction

This programming project focuses on convolution of a grayscale image with the following three convolution kernels:

- Blurring
- Sharpening
- Edge Detection

The program thus developed has been implemented in parallel with Message Passing Interface (MPI) using C++ programming language in the high performance computing cluster of TU Bergakademie Freiberg.

Format of the grayscale image

The images provided are in Netpbm grayscale image format. Quoted as "the lowest common denominator grayscale file format," this format had been chosen due to its ease in coding. Unlike the color images consisting of three color channels, with every pixel displaying the combined information of Red, Green and Blue channels, this format consists of one channel. The images considered in this project are said to be of 8-bit depth, i.e. the values of the pixels lie within the interval $[0, 255]$. The value 0 describes black, whereas 255 describes white. The images provided within this project are of uncompressed PGM data format. Therefore, initially, these values are read into a two-dimensional arrays, which are worked upon. Upon completion, the resultant arrays are written to file as per the specifications, which can then be converted to different formats using ImageMagick software.

2 Convolution kernels

Within the context of image processing, a convolution kernel, or a kernel, refers to a matrix that allows manipulation of values of pixel as per requirements. The procedure, known as convolution, refers to the process of computing a new pixel value from the given pixel value along with its surrounding pixel values. The kernels are specified below.

2.1 Gaussian Blur

The aim of this kernel is to smoothen an image, or reduce the noise or detail within an image. specified below.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

2.2 Sharpen

The aim of this kernel is to enhance the sharpness of the image, can be understood to be the opposite of blur.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

2.3 Edge Detection

The aim of this kernel is to identify the edges present in the image. In the context of image processing, an edge can be understood as the collection of points where the brightness suddenly changes, or has discontinuities.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Then, the respective convolution kernel is then applied to a pixel value $I(x,y)$ to obtain a new pixel value $I^*(x,y)$ where (x,y) denotes the position in the image. Convolution procedure can be shown in a mathematical format as follows.

$$I^*(x,y) = \sum_{i=1}^n \sum_{j=1}^n I(x-i+2, y-j+2)k(i,j)$$

where k represents the chosen 3 x 3 convolution kernel. An important aspect to note is the treatment of pixel values at the boundaries of the image. In this implementation, these are treated periodically, i.e. the pixel value at position $(1,0)$ receives information from $(1,511)$, the opposite border, in order to finish the computation of new pixel value.

3 MPI Parallelization and Communication

For the parallel implementation of this code, MPI library is applied. The pixel values are thus distributed in a way such that each process has its own chunk of rows of pixel values, including the information belonging to the halo. Here, a halo refers to the rows of pixels indicated by dashed lines in the figure. The figure representing this division can be seen here in Fig.1. The code is then executed on the university's high performance computing cluster and the times are tabulated against number of processes to assess scalability.

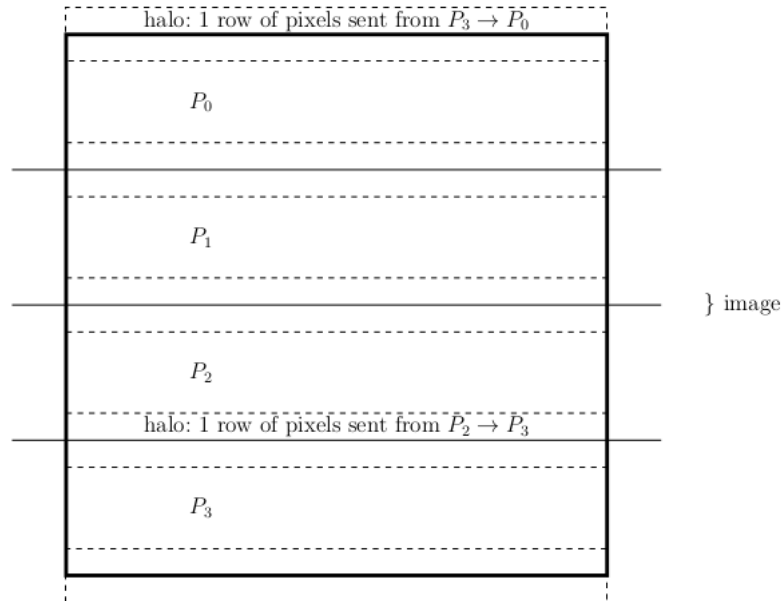


Fig. 1: Distribution of data between processes

4 Results

At the beginning, the results belonging to the 512 x 512 image are presented. Fig.2a presents the original image, Fig.2b shows the image after application of edge detection convolution. Fig.2c and Fig.2d present the result of blurring and application of all three kernels respectively.

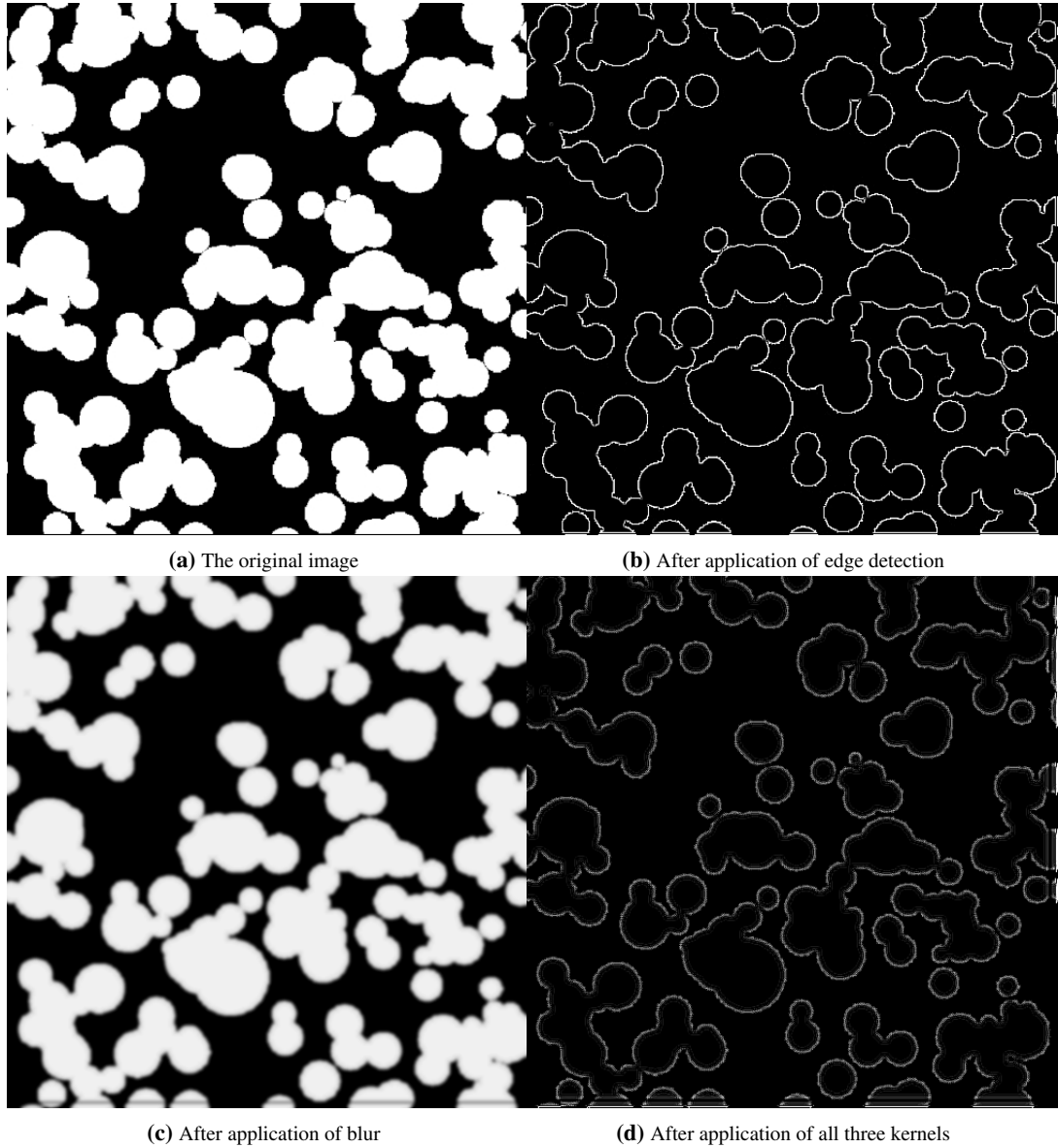


Fig. 2: 512 x 512 grayscale image

The times measured for execution on the cluster are tabulated below. It could be construed from this table that the program exhibits marginal scalability.

#P	With I/O	Without I/O
1	43.8	31.7
2	36.7	28.0
4	28.6	24.6
6	27.8	24.5
8	27.0	24.3
12	26.6	24.0
24	49.1	42.8
36	125.7	61.2

Tab. 1: Measured times for different numbers of processes